

`[SYSTEM BOOT: HACKATHON_BACKEND_GUIDE_v1.0]■

ハッカソン・バックエンド開発プレイブック

API構築、状態管理、チーム連携のための完全ガイド

バックエンドチームの3つの主要任務



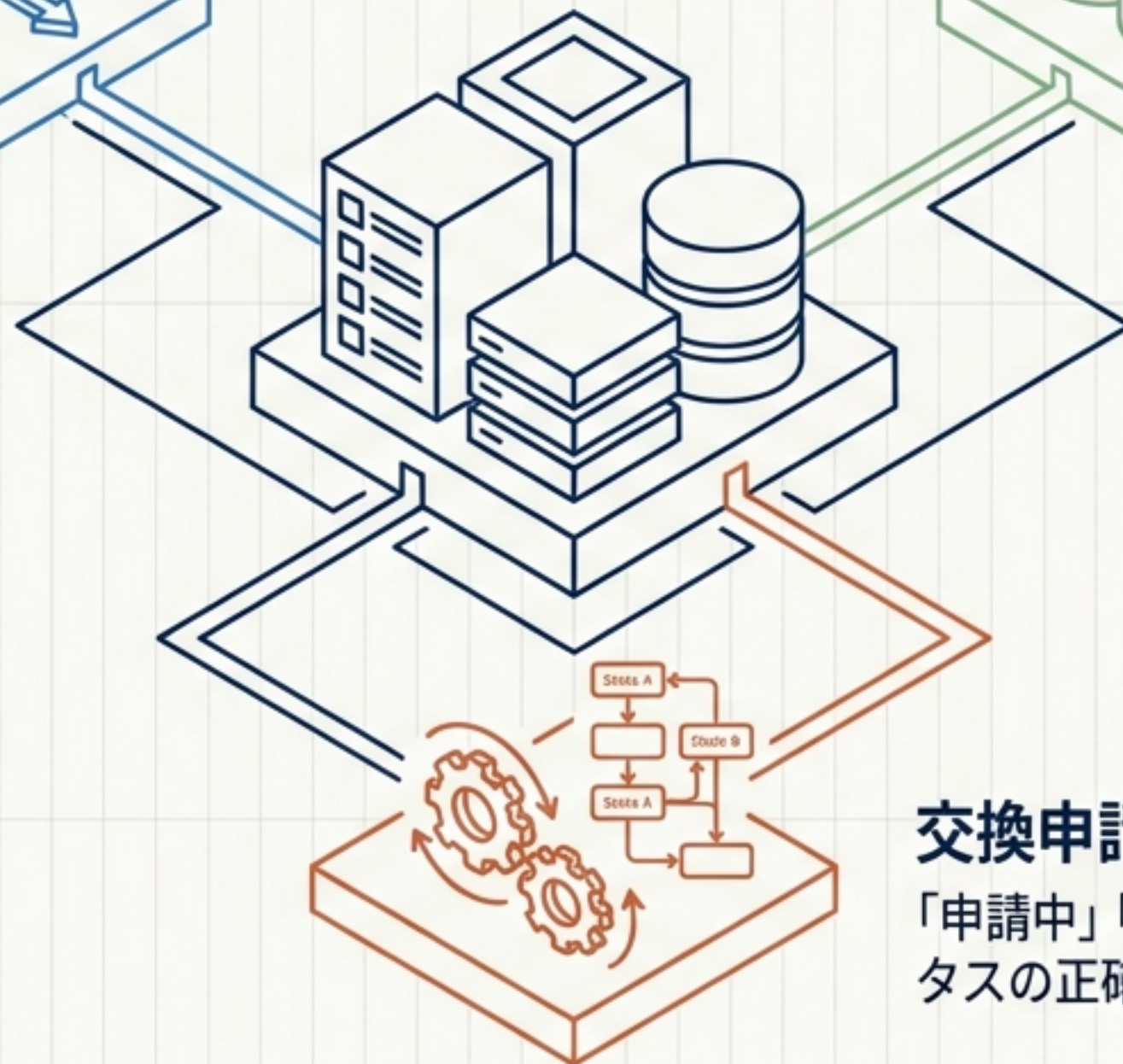
APIサーバーの構築

フロントエンドからのリクエストを受け付け、適切な処理を返す通信のハブ。



データベースの設計と操作

商品データやユーザー情報の永続化と安全な読み書き。



交換申請の状態管理

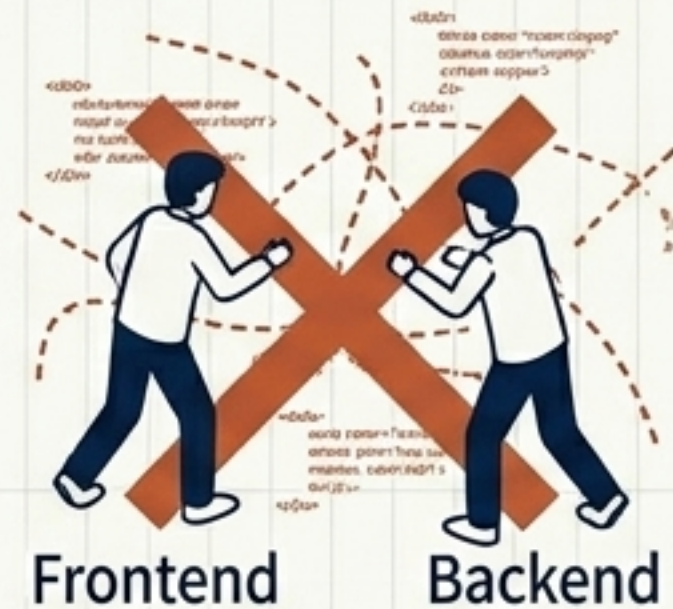
「申請中」「承認」「拒否」など、取引ステータスの正確な遷移と整合性の担保。

APIファースト原則：開発を止めないための絶対ルール

コーディングの前に、必ず `docs/api\_contract.md` にAPI仕様を定義する。

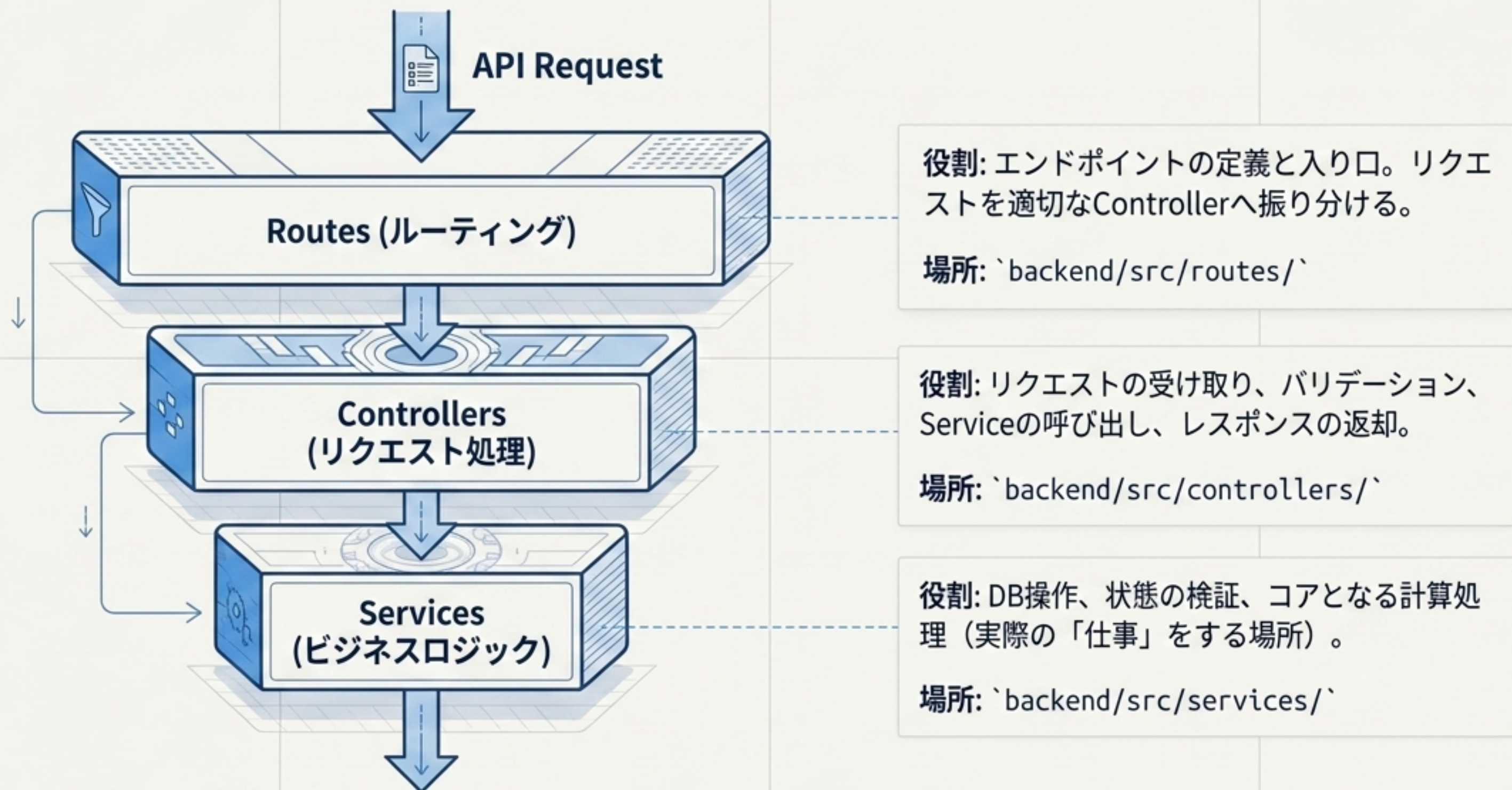
なぜ重要か？

- フロントエンドとバックエンドの約束事:
仕様がFixしていれば、バックエンドの実装完了を待たずにフロントエンドがモックで開発を進められる。
- リーダーの監視: このファイルはチームの生命線であり、リーダーが毎日確認する。



注意: 契約なしのコーディングは
混乱を招く

3層アーキテクチャの解剖図



チーム分担とファイル所有権マトリックス

同じファイルを複数人で同時に触らない（コンフリクト回避の鉄則）。

バックエンド 1（商品ドメイン担当）

Target: Items API

- backend/src/routes/
 - └ itemRoutes.ts
- backend/src/controllers/
 - └ itemController.ts
- backend/src/services/
 - └ itemService.ts

バックエンド 2（交換申請ドメイン担当）

Target: Trade Requests API

- backend/src/routes/
 - └ tradeRequestRoutes.ts
- backend/src/controllers/
 - └ tradeRequestController.ts
- backend/src/services/
 - └ tradeRequestService.ts

ターゲット1: 商品管理 (Items) の実装要件

[GET]

/items

機能: 商品一覧を取得する。



[GET]

/items/:id

機能: 特定の商品詳細を取得する。



[POST]

/items

機能: 新しい商品を出品する。



ターゲット2: 交換申請 (Trade Requests) の実装要件

[GET]

/trade-requests

機能: 交換申請一覧を取得する。

[POST]

/trade-requests

機能: 新しい交換申請を作成する (リクエストパラメータ必須)。

[PATCH]

/trade-requests/:id

機能: 交換申請の状態 (Status) を更新する。

※バックエンドの最重要任務の一つである「状態管理」の要。

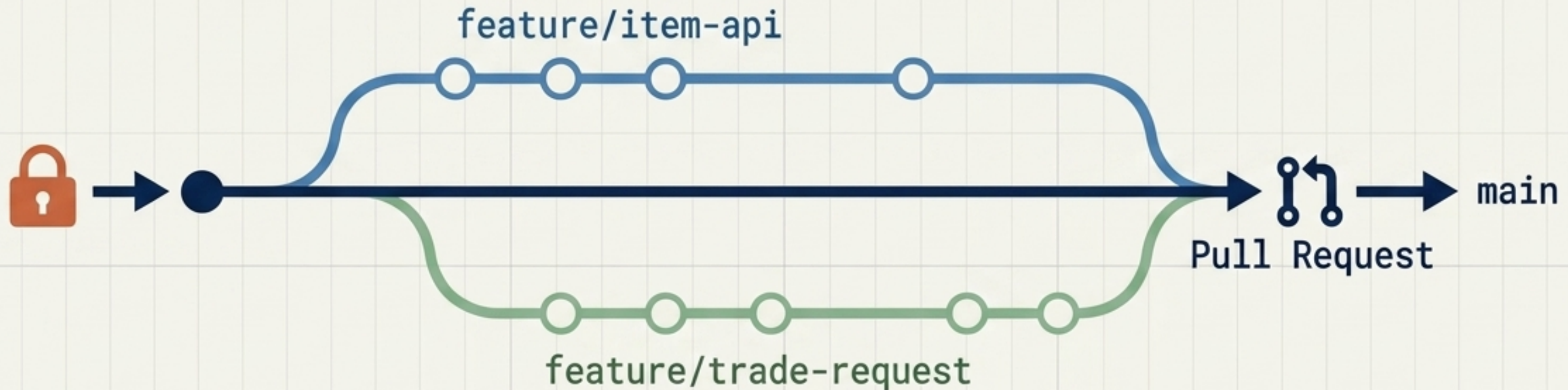
[Pending]

[Approved]

[Rejected]

コンフリクトを防ぐブランチ運用ルール

Core Directive: `main` ブランチには直接作業しない。



1. ブランチの作成

作業時は必ず用途に応じたブランチを切る（例: `feature/item-api`、`feature/trade-request`）。

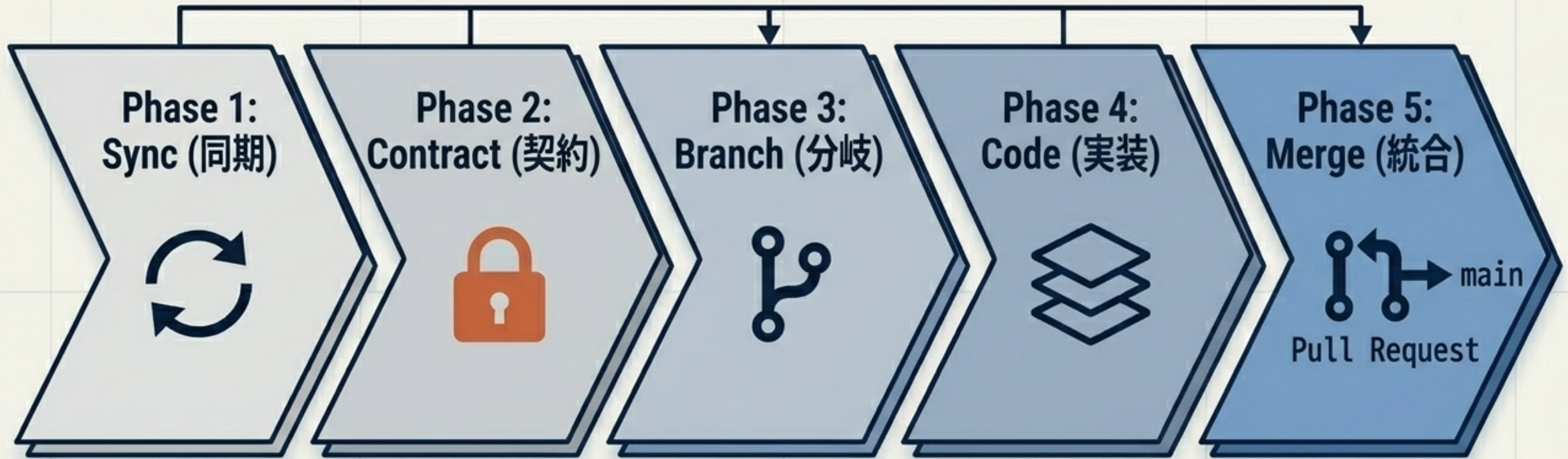
2. 画面・機能ごとの分割

ファイルは機能ごとに明確に分け、他人の担当ファイルには触れない。

3. App.tsxの注意点

変更が衝突しやすいため、作業担当を事前に決めること。

ハッカソン開発のディリーサイクル



`git checkout main`
& `git pull`
`origin main`
で最新状態を取得。

`docs/api\_contract`
mdを確認・更新し、
フロントエンドと合
意。

新しい作業用ブ
ランチを作成。

担当領域の`routes`
→ `controllers`
→ `services`の3
層を実装。

Pull Requestを作
成し、リーダーのレ
ビューを経て`main`
へ統合。